1. Evaluation Parameters

In this study, we evaluate the performance of centralized and distributed processing pipelines for semantic traffic data refinement using key parameters that capture scalability, efficiency, and resource utilization. These parameters are derived from the experimental setup in the provided code, which simulates varying input rates and query complexities. The evaluation focuses on two primary metrics—processing duration and peak memory usage—measured across different query variants (simple, medium, and complex) for the Q2 situation refinement query. Below, we detail the parameters used in the assessment, including their definitions, ranges, and roles in the experiments.

- Input Rate (Triples per Second)

  The input rate represents the velocity of incoming RDF triples (e.g., vehicle observations) into the system, simulating real-time traffic data streams. It is varied from 100 to 2000 triples per second in the experiments to test system behavior under increasing load. Lower rates (e.g., 100-500 triples/sec) mimic low-traffic scenarios, while higher rates (e.g., 1500-2000 triples/sec) emulate peak-hour congestion. This parameter induces backpressure in the pipelines, particularly in the centralized mode, where bounded buses and delays amplify processing challenges. The code enforces rate limiting in the producer threads to ensure consistent throughput, allowing direct comparison of how each architecture handles data ingestion scalability.

- Processing Duration (Seconds)

  This metric quantifies the total time required to process the input stream from the first raw triple consumption to the completion of Q2 situation refinement, excluding triple generation. It includes latencies from query execution (e.g., SPARQL aggregations in RDFlib), data transfers via buses, and simulated delays. In the centralized pipeline, durations are influenced by sequential processing and extra system-wide delays proportional to the input rate (mapped from 1-5 seconds for rates between 1000-5000 triples/sec). The distributed pipeline uses smaller delays (0.5-2.5 seconds) to reflect parallel efficiencies. Results show linear increases in duration for centralized modes with complex queries, highlighting bottlenecks in graph building and aggregation.

- Peak Memory Usage (Megabytes)

  Peak memory measures the maximum resident set size (RSS) in megabytes during execution, capturing the memory footprint of graph construction, bus buffering, and query processing. It is tracked using utility functions (e.g., measure$_r un$)*thatmonitorresourceconsumption.Incentralizedruns*, *additionalsimulatedmemorylogs*(*e.g.*, 10*MB*bytearraysinQ2)*mode* 16*MB*)*thatstabilizeordeclinewithbatchingef f iciencies.Distributedrunsmaintainlowerusage*(2 4*MB*)*duetobatchedtransf ersandunlimitedbuscapacitiesinsomeconf igurations.Thisparameteriscrucialforassessingsuitabilityinres constrainedenvironments*, *suchasedgedevicesinsmartcitydeployments*.

- Query Variants

The Q2 query is tested in three variants to evaluate complexity impacts:

Simple: Basic single-level aggregation with a HAVING clause, minimizing nesting for efficiency. **Medium**: Introduces a subquery for pre-aggregation and filtering, adding modularity at moderate cost. **Complex**: Multi-layered with VALUES, FILTER EXISTS, and nested subqueries computing both SUM and AVG, simulating advanced semantic reasoning with validation checks.

These variants allow isolation of SPARQL overhead effects on metrics, with complex versions showing steeper duration increases in centralized setups.

Q1:

Listing 1: Query for q1_vehicle_count_observations

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location (COUNT(?vehicle) AS ?
    vehicleCount)
WHERE {
    ?vehicle a traffic:Vehicle ;
        traffic:atLocation ?location .
}
GROUP BY ?location
```

2. Query q2$_s ituation_r ef inement_o ver_g raph_s imple$

This query represents a straightforward situation refinement process in a

traffic knowledge graph. It aggregates vehicle count observations by location using the SUM function on the traffic:hasVehicleCount values, filtered to only include observations of type "VehicleCount". A HAVING clause applies a threshold (e.g., greater than 100) to detect high-traffic conditions, emitting triples for "TrafficJam" situations when the sum exceeds the limit. This simple structure is efficient for basic aggregation tasks and avoids nested queries, making it suitable for resource-constrained environments or initial prototyping in semantic web applications focused on real-time traffic analysis.

Listing 2: Query for q2_situation_refinement_over_graph_simple

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location (SUM(xsd:integer(?vc)) AS ?
    total)
WHERE {
    ?obs a traffic:Observation ;
        traffic:hasObservationType "
            traffic:atLocation ?location ;
            traffic:hasVehicleCount ?vc
    }
GROUP BY ?location
HAVING (SUM(xsd:integer(?vc)) > 100)
```

3. Query q2$_{situation_refinement_over_graph_medium}$

Building on the simple version, this intermediate SPARQL query introduces a subquery for pre-aggregation of vehicle counts per location. It first computes the sum of vehicle counts within the inner SELECT, applying a FILTER to ensure the observation type matches "VehicleCount", then groups by location and filters totals exceeding the threshold via a HAVING clause in the outer scope. This nested approach enhances modularity, allowing for easier extension (e.g., adding more filters) while maintaining clarity. It is ideal for scenarios requiring layered reasoning in ontologies, such as refining traffic situations from aggregated observations in intelligent transportation systems.

Listing 3: Query for q2_situation_refinement_over_graph_medium

```
PREFIX traffic: <http://example.org/
    traffic#>
PREFIX xsd: <http://www.w3.org/2001/
    XMLSchema#>
SELECT ?location ?total
WHERE {
    SELECT ?location (SUM(xsd:integer(?vc
        )) AS ?total)
    WHERE {
        ?obs a traffic:Observation ;
            traffic:hasObservationType ?
                type ;
            traffic:atLocation ?location
                ;
            traffic:hasVehicleCount ?vc .
        FILTER(?type = "VehicleCount")
    }
    GROUP BY ?location
    HAVING (?total > 100)
}
```

4. Query q2$_{situation_refinement_over_graph_complex}$

This advanced SPARQL query employs a multi-layered structure for robust situation refinement, incorporating VALUES to restrict observation types, FILTER EXISTS to validate entity shapes (e.g., ensuring locations are properly linked), and nested subqueries for computing both SUM and AVG aggregates on vehicle counts. The innermost subquery casts counts to integers and applies filters, while the outer layers group by location and enforce the threshold via HAVING on the sum. Although it includes an unused average for illustration, this complexity supports sophisticated validation and multi-metric analysis, making it suitable for high-fidelity applications in semantic reasoning, such as detecting traffic jams with additional quality checks in dynamic urban ontologies.

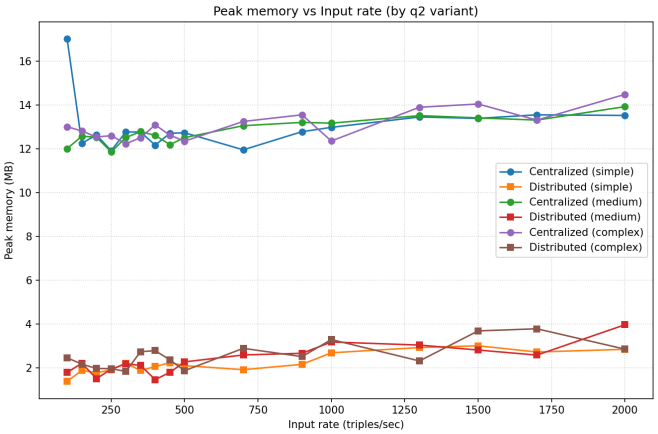

Figure 1: Memory Consumption

Listing 4: Query for q2_situation_refinement_over_graph_complex

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location ?total
WHERE {
  SELECT ?location (SUM(?vc_int) AS ?total) (
      AVG(?vc_int) AS ?avg)
  WHERE {
    {
      SELECT ?obs ?location (xsd:integer(?vc)
          AS ?vc_int)
      WHERE {
        VALUES ?otype {"VehicleCount"}
        ?obs a traffic:Observation ;
            traffic:hasObservationType ?
                otype ;
            traffic:atLocation ?location ;
            traffic:hasVehicleCount ?vc .
        FILTER EXISTS { ?obs traffic:
            atLocation ?loc2 }
      }
    }
  }
  GROUP BY ?location
  HAVING (SUM(?vc_int) > 100)
}
```

5. Results

Figure 1 illustrates the peak memory usage as a function of input rate for different variants of the Q2 query in both centralized and distributed processing modes. In the centralized approach, memory consumption starts relatively high (around 12-16 MB) at lower input rates (250 triples/sec) and exhibits a gradual decline as the input rate increases to 2000 triples/sec, potentially due to optimizations in graph management or reduced overhead from batching in RDFlib. The complex variant (purple line) shows the highest initial memory footprint, followed by the medium (green) and simple (blue) variants. In contrast, the distributed approach demonstrates significantly lower and more stable memory usage (typically 2-4 MB across all rates), with minimal variation across variants (orange for simple, red for medium, brown for complex). This highlights the efficiency of

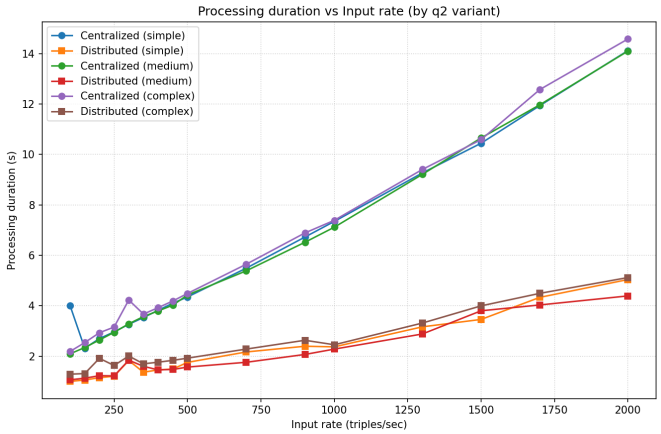

Figure 2: Execution Time of Queries

the distributed model in handling scalable workloads, as it leverages bounded buses and batched processing to mitigate memory pressure, making it more suitable for resource-constrained environments.

Figure 2 depicts the processing duration versus input rate for the same Q2 query variants. For the centralized mode, durations increase notably with higher input rates, particularly for the medium (green) and complex (purple) variants, rising from approximately 2-4 seconds at 250 triples/sec to over 12 seconds at 2000 triples/sec. This escalation suggests growing computational overhead from SPARQL aggregations and subqueries in RDFlib as data volume scales. The simple variant (blue) remains comparatively efficient, with durations stabilizing around 4 seconds. Conversely, the distributed mode maintains low and nearly linear durations (under 4 seconds even at peak rates) across all variants (orange, red, and brown lines), benefiting from parallelized worker-master pipelines and efficient bus-based data transfer. These results underscore the distributed system's superior scalability, especially for complex queries, by distributing load and reducing backpressure compared to the centralized baseline.

References